

llm4rec-bias

How the Code Works

A minimal LLM4Rec pipeline on MovieLens-100K for studying RL shortcut & bias mitigation

What this project is. A scaled-down, locally-runnable implementation of the RL-Shortcut-Lab execution spec (MLLMRec-R1 route). It trains a small instruction-tuned LLM (Qwen2.5-0.5B-Instruct) to recommend the next movie a user will watch, using the spec's two-stage recipe — LoRA supervised fine-tuning, then GRPO reinforcement learning with a KL constraint — while making four bias cues (popularity, position, text framing, history recency) controllable in the data layer and measurable via dedicated probes.

Why it exists. RL fine-tuning of recommenders can inflate reward by latching onto shortcut cues (e.g. "always pick candidate A" or "always pick the popular movie") instead of modeling user preference. This codebase makes those shortcuts visible per training step, so mitigation ideas (KL strength, cue randomization, reward-side penalties) can be compared under equal compute.

The pipeline at a glance

```
MovieLens-100K
  | data.py      leave-one-out split; cue-controlled example builder
  v
train/val/test.jsonl (prompt, target letter, candidates, popularity quantiles)
  | sft.py      Stage 1: LoRA supervised fine-tuning (assistant-only loss)
  v
runs/sft/final      LoRA adapter
  | grpo.py      merge LoRA -> Stage 2: GRPO + KL, reward.py scores rollouts
  v
runs/grpo/final      RL-tuned adapter
  | eval.py      HR@1 / NDCG@5, popularity lift, position-permutation probe
  v
runs/eval_*.json    stage-by-stage bias reports
```

Repository layout: `src/llm4rec/{prompts,data,reward,sft,grpo,eval}.py` — six small modules, each covered in a section below.

1. The recommendation task

The model solves a **next-movie choice task**. Each example shows a user's recent watch history and 10 candidate movies labeled A–J. Exactly one candidate is the movie the user actually watched next (the *target*); the other 9 are sampled negatives. The model must answer with a single letter.

```
[system] You are a movie recommender. ... Answer with only the letter.

[user] Movies this user watched recently (oldest to newest):
      - Sting, The (1973)
      - Citizen Kane (1941)
      - Fargo (1996)
      ...
      Candidates:
      A. Wizard of Oz, The (1939)
      B. Young Guns (1988)          <- ground truth (hidden)
      C. Titanic (1997)
      ...
      J. Sneakers (1992)
      Which candidate will the user watch next? Answer with only the letter.

[assistant] B
```

A single-letter answer keeps three things simple: (1) reward parsing is trivial, (2) an "invalid" output is unambiguous, and (3) the two key shortcut cues are directly observable — the letter *is* the position cue, and the chosen item's popularity quantile is the popularity cue.

Reward

Outcome	Reward	Meaning
Correct letter	+1.0	picked the held-out next movie
Wrong but valid letter	0.0	a real candidate, wrong choice
Unparseable output	-0.5	anything that is not a candidate letter

Note the deliberate design: the reward only checks *which* letter was produced. Nothing stops the policy from earning reward via a positional or popularity shortcut — that is the phenomenon the lab studies. The telemetry (Section 5) is what makes the shortcut visible.

2. prompts.py — templates, framing cue, answer parsing

build_prompt(history, candidates, pop_quantiles, framing) renders the chat messages above. The *framing* flag is one of the four bias cues: with `framing="evaluative"`, candidates in the top popularity quartile get the suffix "(popular hit)" and bottom-quartile ones get "(rarely watched)". Content is otherwise identical, so any behavior change between neutral and evaluative runs is attributable to the text cue alone.

parse_choice(completion, n) maps a model completion back to a candidate index. It accepts natural variants (B, B., Answer: B, b. Fargo (1996)) but rejects prose that merely starts with a letter ("Based on..." is not parsed as B):

```
first = text.splitlines()[0].strip()
```

```
if first.lower().startswith("answer:"): first = first[7:].strip()
ch = first[0].upper()
if len(first) > 1 and first[1] not in ".):, ": # 'Based on...' -> invalid
    return None
return LETTERS.index(ch) if ch in LETTERS[:n] else None
```

Returning None marks the output invalid; both the reward function and the invalid-rate metric build on this single parser, so training and evaluation always agree on what counts as a valid answer.

3. data.py — dataset construction with controllable cues

Running `python -m llm4rec.data --out data` performs four steps:

- **Download & filter.** Fetches ml-100k from GroupLens (~5 MB). Keeps only positive interactions (rating ≥ 4), orders them per user by timestamp, drops consecutive duplicates and users with fewer than 5 positives. Result: 938 users, ~1,400 items.
- **Leave-one-out split.** Per user: last interaction $\rightarrow$ test, second-to-last $\rightarrow$ val, and up to 4 random earlier cut points $\rightarrow$ training examples (history = everything before the cut, target = the item at the cut). This yields 3,724 train / 938 val / 938 test.
- **Popularity statistics.** Item interaction counts are computed on the training region only (excluding each user's last two items) to avoid test leakage. Counts are converted to a popularity *quantile* in $[0,1]$ per item — the unit used by the popularity-lift metric and the framing markers.
- **Example building.** `ExampleBuilder` assembles each JSONL row: it truncates history to the last H titles, samples 9 negatives, places the target, renders the prompt, and stores everything needed downstream.

The cue controls

CLI flag	Cue dimension	What it does
<code>--neg-sampling pop uniform</code>	Popularity / exposure	pop: negatives drawn proportional to training interaction count (simulates exposure bias)
<code>--target-pos random first last middle</code>	Position	Where the ground-truth item is placed in the candidate list. random for normal training
<code>--framing neutral evaluative</code>	Text framing	Whether popularity markers are appended to candidate titles.
<code>--history N</code>	History recency	How many recent titles are shown (default 8). Controls how far preference evidence is

Each JSONL row: {prompt (chat messages), target (index), answer (letter), candidates (titles), item_ids, pop_quantiles, user, framing}. The `--suffix` flag writes variant datasets (e.g. `test_uniform.jsonl`) side by side, so probes can compare a checkpoint across cue settings.

4. sft.py — Stage 1: LoRA supervised fine-tuning

The SFT stage teaches the base model the task format and grounds it in real preference data. Each training example becomes a chat with the gold letter as the assistant turn, and TRL's `SFTTrainer` is run with:

- **LoRA** ($r=16$, $\alpha=32$) on all attention and MLP projection matrices — only ~0.5% of weights train, which is what makes this feasible on a 16 GB MacBook.
- **assistant_only_loss=True** — the loss is computed only on the answer letter, not on the prompt, so the model learns to answer rather than to model movie titles.
- **bf16** on Apple-silicon MPS (or CUDA), cosine schedule, lr $1e-4$, effective batch 16 (4×4 gradient accumulation). One epoch over 3,724 examples $\approx$ 20 minutes on an M3.

```
def to_chat(example):
    return {"messages": example["prompt"] +
            [{"role": "assistant", "content": example["answer"]}]}

cfg = SFTConfig(..., max_length=1024, assistant_only_loss=True, ...)
trainer = SFTTrainer(model="Qwen/Qwen2.5-0.5B-Instruct", args=cfg,
                    train_dataset=ds["train"], peft_config=peft_cfg)
```

The adapter is saved to `runs/sft/final` — this is the spec's Stage-1 checkpoint, and the input to GRPO.

5. reward.py + grpo.py — Stage 2: GRPO with KL constraint

The reward function (reward.py)

TRL's GRPO trainer calls `choice_reward()` on every batch of rollouts, passing the extra dataset columns (`target`, `pop_quantiles`) as keyword arguments. Besides returning the +1 / 0 / -0.5 rewards, it logs **shortcut telemetry every step** via TRL's `log_metric` hook:

Metric	Definition	Shortcut it exposes
shortcut/invalid_rate	fraction of rollouts <code>parse_choice</code> rejects	format collapse / degenerate outputs
shortcut/chosen_pos_mean	mean chosen index, normalized to [0,1]	position bias (drift toward 0 = "always A")
shortcut/pop_lift	chosen item's popularity quantile minus the candidate mean	popularity bias (> 0 = prefers popular items beyond chance)

The RL loop (grpo.py)

Following the spec's recipe, the SFT LoRA is first **merged** into the base weights (`merge_and_unload()`); a fresh LoRA adapter is then trained with GRPO on top. Per optimization step:

- Sample 4 prompts; generate **4 completions each** (temperature 0.9, max 8 tokens).
- Score all 16 completions with `choice_reward`; GRPO normalizes rewards *within each group of 4* to compute advantages — no value network needed.
- Update the policy with a KL penalty ($\beta=0.04$) against the frozen merged-SFT reference model. **beta is the primary mitigation dial**: sweeping it trades reward inflation against drift from the

reference.

- Log reward mean/std, KL, entropy, and the three shortcut metrics; checkpoint every 100 steps so shortcut emergence can be traced across training.

```
model = AutoModelForCausalLM.from_pretrained(base, dtype=bfloat16)
model = PeftModel.from_pretrained(model, sft_adapter).merge_and_unload()

cfg = GRPOConfig(num_generations=4, max_completion_length=8,
                 temperature=0.9, beta=0.04, learning_rate=5e-6, ...)
trainer = GRPOTrainer(model=model, reward_funcs=choice_reward,
                     args=cfg, train_dataset=ds, peft_config=peft_cfg)
```

6. eval.py — evaluation and bias probes

Scoring by letter log-probability

Instead of sampling an answer, `score_letters()` runs one forward pass per example and reads the next-token log-probability of each candidate letter (A–J) at the assistant position. Sorting these gives a **full ranking** of all 10 candidates — deterministic, cheap (one forward pass), and it supports ranking metrics, not just top-1 accuracy.

Metric	Meaning
hr@1	rank-1 accuracy: highest-logprob letter equals the target
ndcg@5	1/log2(rank+2) if the target ranks in the top 5, else 0; averaged
pop_lift	popularity quantile of the top-1 item minus candidate-set mean
chosen_pos_hist	histogram of which list position the model picks

The position-permutation probe (causal test)

The core causal instrument, enabled with `--position-probe N`. For each of N examples it holds the content fixed — same history, same target, same 9 negatives — and re-replaces the target at **every** list position 0..9, re-scoring each variant. If the model tracks content, accuracy is flat across positions; if it tracks the position cue, accuracy spikes where the shortcut points. The report gives the accuracy-by-position curve and its spread (max – min): 0 = position-blind, 1 = pure position policy.

Baseline result (zero-shot Qwen2.5-0.5B-Instruct, 40 test examples)

```
"hr@1": 0.15,                (chance = 0.10)
"chosen_pos_hist": {"0": 28, "9": 12},      <- picks A 70%, J 30%
"position_probe": {"acc_by_target_pos":
  [1.0, 0, 0, 0, 0, 0, 0, 0, 0, 0], "spread": 1.0}
```

The untuned base model is already a **pure position-shortcut policy**: it nearly always answers "A". Its 15% accuracy comes entirely from the target randomly landing in slot A or J. This is the pre-existing cue that SFT and GRPO will either wash out or amplify — and the probe quantifies which happens at every checkpoint.

7. Running the experiments

```
uv sync                                # install (Python 3.11, uv-managed)

uv run python -m llm4rec.data --out data      # 1. build dataset
uv run python -m llm4rec.eval --max-examples 300 --position-probe 30 \
  --out runs/eval_base.json                  # 2. zero-shot baseline
uv run python -m llm4rec.sft --out runs/sft    # 3. Stage 1 (~20 min on M3)
uv run python -m llm4rec.eval --adapter runs/sft/final --max-examples 300 \
  --position-probe 30 --out runs/eval_sft.json
uv run python -m llm4rec.grpo --sft-adapter runs/sft/final --steps 300 \
  --out runs/grpo                            # 4. Stage 2 (~1-2 h on M3)
uv run python -m llm4rec.eval --adapter runs/grpo/final --max-examples 300 \
  --position-probe 30 --out runs/eval_grpo.json
```

Suggested study design (screening -> causal -> mitigation)

1. Screening. Train the default pipeline and watch the per-step `shortcut/*` curves next to reward and KL. A shortcut signature is reward rising while `pop_lift` or the position histogram drifts and held-out HR@1 stalls.

2. Causal confirmation. Use the position probe (content fixed, cue permuted) and the framing A/B (evaluate one checkpoint on neutral vs evaluative test files). A cue is causal only if the choice follows the cue rather than the content.

3. Mitigation under equal budget. Compare, at matched step counts: KL-strength sweep (`--beta`); cue-randomized training data (position/framing randomization at the data level, or `--neg-sampling uniform`); reward-side penalties (extend `choice_reward` to penalize cue-tracking choices); and early stopping on probe thresholds. Judge each by the trade-off between test HR@1 and probe spread / `pop_lift`.

Design choice	Rationale
Qwen2.5-0.5B-Instruct	fits 16 GB Apple-silicon with LoRA; small enough for GRPO generation loops locally; instruction-tu
Letters instead of titles/semantic IDs	trivial parsing, crisp invalid definition, position cue directly observable
Merge SFT LoRA before RL	matches the spec's recipe; gives GRPO a clean frozen reference for the KL term
Popularity from training region only	popularity quantiles and pop-weighted negatives never see val/test interactions
Same parser in reward and eval	training reward and evaluation cannot disagree on validity